

BRD và ERD/Data Model cho nền tảng WebApp Task Processing bằng Go + gocron

Kết luận kiến trúc và định hướng thiết kế

Tại ngày **17/08/2026**, bản phát hành mới nhất của `go-co-op/gocron/v2` là **v2.22.0**, phát hành ngày 09/07/2026. Nhánh v2 là nhánh được duy trì; v1 đã deprecated và không còn nhận security patches. Vì vậy thiết kế mới nên cố định trên `github.com/go-co-op/gocron/v2`, không dùng API của v1. ¹

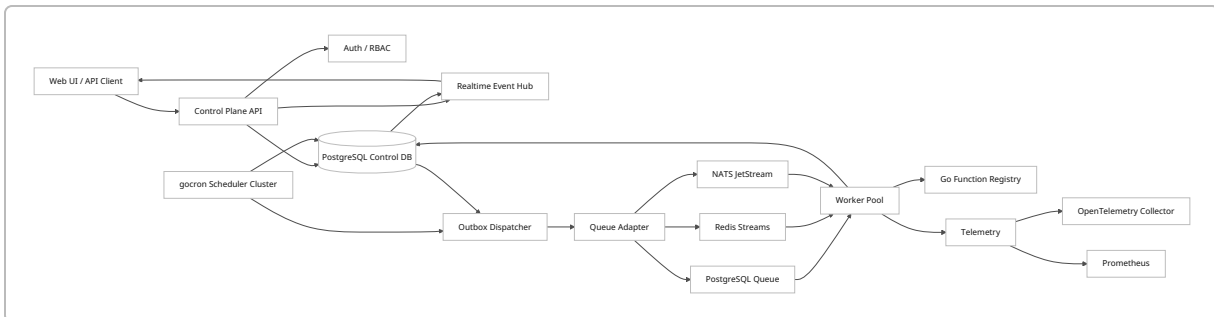
Điểm quan trọng nhất của BRD này là: **không nên coi gocron là một durable task queue**. Tài liệu chính thức mô tả `gocron` là thư viện dùng để chạy Go functions theo các lịch định trước. Nó cung cấp Cron, duration, one-time jobs, singleton execution, concurrency limits, distributed locking/election, lifecycle listeners, monitor hooks, `RunNow`, start/stop scheduler và các tính năng scheduling khác; nhưng API hiện tại không có first-class primitives cho retry policy, dead-letter queue, job priority, per-queue pause/resume, durable batch queue hay workflow DAG. ²

Thậm chí `LimitModeWait` của `gocron` chỉ cung cấp hàng đợi khi scheduler đã chạm concurrency limit; tài liệu cảnh báo execution order không được đảm bảo, một job chạy thường xuyên có thể tích tụ nhiều executions trong wait queue, và không nên dùng chế độ này khi tốc độ sinh việc lớn hơn khả năng xử lý. Vì thế `LimitModeWait` **không nên được sử dụng thay cho business job queue**. ³

Kiến trúc đề xuất là tách rõ ba khái niệm:

Layer	Trách nhiệm
Scheduler layer — <code>gocron</code>	Quyết định <i>khi nào</i> tạo một execution: cron, interval, one-time, recurring maintenance
Queue layer	Quyết định <i>execution nào được worker nhận tiếp theo</i> : durability, priority, rate limit, pause, retry, lease, DLQ
Worker layer	Quyết định <i>function được chạy như thế nào</i> : middleware, timeout, idempotency, execution, logging, metrics

Với mô hình này, một `gocron` task **không trực tiếp làm business processing dài hạn**. Nó chủ yếu tạo một durable `job_run` rồi enqueue execution đó. Worker mới là thành phần thực thi function. Đây là ranh giới quan trọng nhất giúp hệ thống có thể retry, scale, đổi queue backend, có DLQ và không mất job khi scheduler process restart.



Đối với HA scheduler, `gocron` có cả `WithDistributedLocker` và `WithDistributedElector`. Tuy nhiên tài liệu cũng lưu ý distributed locker không tự đồng bộ thời điểm chạy giữa nhiều scheduler và vẫn chịu ảnh hưởng của clock skew; tài liệu khuyến nghị cân nhắc Elector nếu đặc tính đó không chấp nhận được. Vì scheduler ở kiến trúc này chỉ cần phát sinh một durable execution, lựa chọn ưu tiên là **leader election cho scheduler**, kết hợp unique occurrence key ở database để chống duplicate trigger. ⁴

`gocron-ui` chính thức có thể là reference implementation tốt cho prototype vì đã có WebSocket realtime updates, multi-scheduler, manual trigger, schedule preview và REST API. Tuy nhiên repository hiện ghi rõ job không thể được tạo từ UI vì Go functions phải được compile vào binary; authentication không được package cung cấp sẵn và default CORS cho phép mọi origin. Do đó với yêu cầu Scoped UI, Authentication, Audit, DLQ và Queue Management của hệ thống này, **nên xây control-plane UI riêng**, có thể tham khảo cách `gocron-ui` sử dụng WebSocket và introspect scheduler. ⁵

BRD, phạm vi và yêu cầu nghiệp vụ

Business objective đề xuất là xây dựng một nền tảng internal/enterprise Task Processing cung cấp một abstraction thống nhất để nhiều ứng dụng đăng ký Go functions, chạy functions theo yêu cầu hoặc lịch Cron, đưa executions vào durable queues, xử lý bởi worker pools có khả năng scale horizontally, và vận hành toàn bộ lifecycle thông qua Web UI/API.

BRD này giả định các cột trống trong bảng ban đầu của yêu cầu là ma trận capability/tier chưa có tên rõ ràng. Vì vậy các feature được xem là **target capabilities của sản phẩm**, còn Community/Support/SLA được xem như phần product packaging thay vì runtime behavior.

Actors chính được đề xuất gồm Platform Administrator quản trị backend/tenant/security; Operator vận hành queue, retry, DLQ và workers; Developer đăng ký function/job/schedule; Viewer chỉ xem dashboard/history; API Client tạo job bằng REST API; Scheduler tạo executions theo lịch; Worker thực thi functions.

Không nằm trong scope của platform là upload arbitrary Go source code qua Web UI và compile động trong production. `gocron-ui` cũng phản ánh constraint tương tự: scheduled functions là Go code đã compile. ⁵ Function trong platform nên được deploy cùng worker binary hoặc plugin/service đã được kiểm soát, sau đó worker đăng ký handler name vào `FunctionRegistry`.

Ma trận BRD đề xuất:

Requirement	Yêu cầu	Priority	Target behavior
BR-QUEUE-01	Multiple named queues	Must	Tạo nhiều queue độc lập theo project/workload
BR-QUEUE-02	Multiple backends	Must	PostgreSQL baseline; Redis Streams/NATS JetStream adapters
BR-JOB-01	Immediate jobs	Must	API enqueue ngay
BR-JOB-02	Scheduled jobs	Must	Cron/interval/one-time qua <code>gocron</code>
BR-JOB-03	Function registry	Must	Function name → compiled Go handler
BR-JOB-04	Parameter payload	Must	JSON payload + schema/version metadata
BR-JOB-05	Idempotency	Must	Client key + unique job execution
BR-RETRY-01	Automatic retry	Must	Fixed/exponential backoff + jitter
BR-RETRY-02	Error classification	Must	Retryable/non-retryable categories
BR-DLQ-01	Dead Letter Queue	Must	Exhausted jobs chuyển DLQ
BR-DLQ-02	Replay DLQ	Must	Requeue một/nhiều DLQ entries
BR-PRIO-01	Job priority	Must	Multiple priority bands, anti-starvation
BR-CONC-01	Worker concurrency	Must	Global/queue/function limits
BR-RATE-01	Rate limiting	Must	Queue/function/tenant token bucket
BR-BATCH-01	Batch processing	Must	Handler nhận một batch
BR-BULK-01	Bulk inserts	Must	Atomic control-record creation + eventual backend publication
BR-QUEUE-03	Pause/resume	Must	Soft pause, resume, drain
BR-MW-01	Middleware	Must	Standard + custom middleware chain
BR-UI-01	Realtime UI	Must	WebSocket/SSE state changes
BR-UI-02	Scoped UI	Must	Tenant/project/queue scope
BR-SEC-01	Authentication	Must	Enterprise identity/OIDC integration
BR-SEC-02	RBAC	Must	Admin/Operator/Developer/Viewer
BR-SEC-03	Encryption at rest	Must	DB/storage encryption + sensitive payload encryption
BR-OBS-01	Job history	Must	Searchable executions + attempts
BR-OBS-02	Structured logs	Must	Correlated by run/attempt/trace

Requirement	Yêu cầu	Priority	Target behavior
BR-OBS-03	Metrics	Must	Queue, worker, scheduler, latency, retry, DLQ
BR-OBS-04	Error alerts	Must	DLQ growth/failure ratio/queue age
BR-OBS-05	Custom telemetry	Should	OpenTelemetry exporters/Collector
BR-WF-01	Workflow DAG	Should	Dependencies and conditional progression
BR-WF-02	Workflow retries	Should	Step-level retry/failure policy
BR-AUD-01	Audit trail	Must	Immutable administrative action history
BR-RET-01	Retention policies	Must	Per environment/tier retention
BR-EXT-01	Backend SDK	Should	Implement custom queue adapters
BR-EXT-02	Middleware SDK	Should	Implement custom handler middleware

Các scheduling capabilities nên expose được phần lớn primitives của `gocron`: Cron expressions có năm fields hoặc sáu fields khi bật seconds, scheduler/job timezone, duration, daily, weekly, monthly và one-time schedules. `gocron` cũng hỗ trợ `WithStartAt`, `WithStopAt`, `RunNow`, limited runs và DST policy trong v2 hiện tại. ⁶

Một khác biệt domain quan trọng trong BRD là:

Job Definition ≠ Schedule ≠ Job Run ≠ Job Attempt.

`Job Definition` mô tả function và policies. `Schedule` mô tả thời điểm sinh run. `Job Run` là một logical execution duy nhất. `Job Attempt` là một lần worker thật sự thử xử lý run. Tách bốn đối tượng này là điều kiện cần để lịch Cron có lịch sử nhiều executions và mỗi execution có nhiều retries.

Các support/SLA requirements trong bảng ban đầu nên được lưu trong BRD nhưng không trộn với runtime:

Product capability	Community	Team/Business	Enterprise
GitHub/community	Có	Có	Có
Email/chat	Không bắt buộc	Có	Có
Priority response	Best effort	24h target	2h target
Dedicated account management	Không	Optional	Có
SLA	Không	Optional	Contractual
Custom feature request	Best effort	Prioritized	Priority
Collaborative coding	Không	Optional	Có
Custom telemetry integration	Community API	Có	Có

Các con số SLA trong bảng trên nên được xem là **đề xuất product policy**, không phải năng lực kỹ thuật tự động được đảm bảo bởi `gocron`.

Kiến trúc xử lý, backend abstraction và execution semantics

Control Plane nên giữ **PostgreSQL làm source of truth cho metadata và execution history**, ngay cả khi execution queue thực tế nằm trên Redis hoặc NATS. Cách này tránh việc mỗi backend tự định nghĩa một model khác nhau và cho phép Web UI/API query nhất quán.

Một execution đi qua luồng đề xuất:

```
Schedule/API
  ↓
Create JobRun
  ↓
Insert Outbox Event
  ↓
COMMIT
  ↓
Outbox Dispatcher
  ↓
QueueBackend.Enqueue(run_id)
  ↓
Worker Reserve
  ↓
Middleware
  ↓
Handler
  ↓
Success → ACK
      or
Failure → RetryWait → Requeue
      or
Attempts exhausted → DLQ
```

Transactional outbox là thành phần nên có ngay từ đầu. API/scheduler ghi `job_runs` và `outbox_events` trong cùng database transaction. Dispatcher sau đó publish ra queue backend. Nhờ đó không xuất hiện trạng thái nguy hiểm kiểu “database đã lưu JobRun nhưng process chết trước khi publish queue”. Đây là quyết định kiến trúc của BRD, không phải feature có sẵn của `gocron`.

Queue abstraction Go nên nhỏ và dựa trên semantics thay vì API cụ thể của vendor:

```
type QueueBackend interface {
    Enqueue(ctx context.Context, msg Message) error
}
```

```

    EnqueueBatch(ctx context.Context, msgs []Message) error

    Reserve(ctx context.Context, req ReserveRequest) ([]Delivery, error)

    Ack(ctx context.Context, delivery Delivery) error
    Nack(ctx context.Context, delivery Delivery, reason error) error

    Depth(ctx context.Context, queue string) (QueueStats, error)
    Health(ctx context.Context) error

    Capabilities() Capabilities
}

```

Capabilities nên khai báo rõ:

```

type Capabilities struct {
    NativePriority bool
    NativeDelay    bool
    NativeDLQ      bool
    NativePause    bool
    AtomicBatch    bool
    ConsumerGroups bool
}

```

Không nên buộc adapter phải giả vờ tất cả backend có cùng primitive. Control Plane sẽ emulate feature khi backend không có native capability.

PostgreSQL backend là lựa chọn hợp lý cho MVP. PostgreSQL hỗ trợ

`SELECT ... FOR UPDATE ... SKIP LOCKED`; tài liệu chính thức nói rõ `SKIP LOCKED` tạo một view không nhất quán nên không phù hợp cho general-purpose querying, nhưng có thể dùng để tránh lock contention khi nhiều consumers truy cập một queue-like table. PostgreSQL cũng đề cập `SKIP LOCKED` trong các batch-update patterns khi cần tránh contention. ⁷

Postgres queue có thể claim theo pattern:

```

WITH picked AS (
    SELECT id
    FROM job_runs
    WHERE queue_id = $1
        AND status = 'QUEUED'
        AND available_at <= now()
    ORDER BY priority DESC, available_at ASC, id ASC
    FOR UPDATE SKIP LOCKED
    LIMIT $2
)

```

```

UPDATE job_runs AS j
SET
    status          = 'RESERVED',
    lease_owner     = $3,
    lease_expires_at = now() + $4::interval,
    reserved_at     = now()
FROM picked
WHERE j.id = picked.id
RETURNING j.*;

```

Row lock chỉ nên giữ trong transaction rất ngắn để chuyển trạng thái sang `RESERVED`; không nên giữ database transaction mở suốt thời gian handler thực thi. PostgreSQL row locks giữ đến khi transaction kết thúc, nên lease model sau commit phù hợp hơn cho long-running tasks. ⁸

`LISTEN/NOTIFY` có thể dùng như **wake-up hint** để giảm polling latency: PostgreSQL cho phép sessions `LISTEN` một channel và `NOTIFY` gửi asynchronous notification tới listeners; dữ liệu đầy đủ vẫn nằm ở table. ⁹ Không nên coi `NOTIFY` là durable queue; database rows vẫn là source of truth.

Redis Streams backend phù hợp khi cần throughput và worker scaling cao hơn. Redis Streams consumer groups cung cấp pending entries và acknowledgment; tài liệu Redis mô tả mô hình là at-least-once delivery. `XAUTOCLAIM` cho phép consumer khỏe mạnh claim các pending entries đã idle sau khi worker cũ crash, và Streams hỗ trợ bounded retention/replay. ¹⁰

NATS JetStream backend là lựa chọn tốt cho môi trường event-driven. JetStream cung cấp persistence và at-least-once delivery; server giữ consumer progress, message không được ACK đúng hạn sẽ được redeliver, và cùng một consumer có thể được nhiều clients chia sẻ để phân phối workload. Pull consumers hỗ trợ fetch batch hoặc continuous consumption. ¹¹

Vì Redis Streams và NATS JetStream đều có at-least-once semantics, platform **không nên quảng cáo exactly-once processing**. Worker có thể hoàn thành side effect rồi crash trước ACK, khiến cùng execution được redeliver. Contract đúng phải là:

At-least-once execution + idempotent processing.

Mỗi run nên có `idempotency_key`, `job_run_id` và correlation ID. Handler thực hiện non-idempotent side effects như charge payment, gửi notification hoặc cập nhật hệ thống ngoài phải sử dụng những keys này để deduplicate.

Về priority, không nên phụ thuộc vào một global integer queue ordering mà mọi backend phải implement giống hệt. Đề xuất chuẩn hóa thành năm priority bands:

```

CRITICAL
HIGH
NORMAL

```

LOW
BULK

Trong PostgreSQL có thể map band thành `smallint priority` và `ORDER BY priority DESC`. Với backend không hỗ trợ global priority tự nhiên, adapter có thể dùng nhiều stream/subject và worker áp dụng weighted polling, ví dụ `8:4:2:1:1`, để tránh starvation.

Queue pause nên là **control-plane semantic**, không phụ thuộc backend. `PAUSED` nghĩa là workers không reserve job mới nhưng in-flight jobs được hoàn thành. `DRAINING` nghĩa là không nhận enqueue mới và tiếp tục xử lý backlog. `DISABLED` nghĩa là từ chối cả enqueue lẫn reserve. `gocron` có `StopJobs()` để stop toàn scheduler rồi `Start()` lại, nhưng đây là global scheduler pause, không phải per-business-queue pause. ¹²

HA Scheduler nên dùng:

```
Scheduler Replica A ┐
Scheduler Replica B ┤ leader election → active gocron scheduler
Scheduler Replica C ┘
```

Active scheduler load enabled schedules từ database và register chúng vào `gocron`. Khi schedule thay đổi, Control Plane phát `schedule.changed`; leader chạy `Scheduler.Update()` hoặc recreate mapping. `gocron` hiện hỗ trợ update mà giữ nguyên Job ID và hỗ trợ distributed elector. ¹³

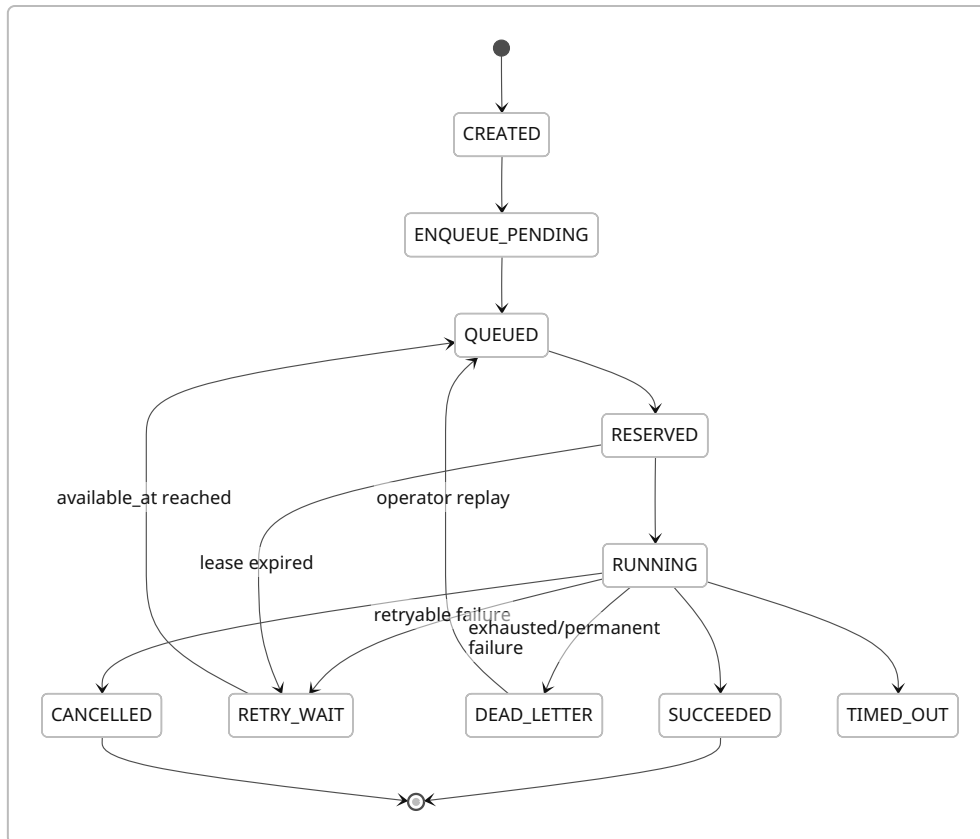
Ngoài election, mỗi occurrence cần database uniqueness:

```
UNIQUE(schedule_id, scheduled_for)
```

Như vậy kể cả scheduler failover hoặc clock anomalies tạo hai trigger cùng một occurrence, database chỉ cho phép một logical `job_run`.

Functional specification, retry, concurrency và workflow

Lifecycle được đề xuất:



Retry policy không nên được implement bằng việc yêu cầu `gocron` schedule lại từng lỗi. Current `gocron` API có event listener như `AfterJobRunWithError`, nhưng package documentation không cung cấp first-class retry-policy API hoặc DLQ API. Retry cần thuộc Queue/Worker domain. ¹⁴

Retry policy đề xuất:

Field	Example
<code>max_attempts</code>	5
<code>strategy</code>	EXPONENTIAL
<code>initial_delay_ms</code>	1000
<code>multiplier</code>	2.0
<code>max_delay_ms</code>	300000
<code>jitter_pct</code>	20
<code>retry_timeout</code>	true
<code>retry_rate_limited</code>	true
<code>retry_dependency_error</code>	true

Field	Example
<code>retry_validation_error</code>	<code>false</code>

Base delay của attempt thứ `n`:

$$delay_n = \min(maxDelay, initialDelay \times multiplier^{n-1})$$

Sau đó thêm jitter ngẫu nhiên trong range policy đã cấu hình.

Worker cần classify lỗi ít nhất thành:

```
TRANSIENT
RATE_LIMITED
DEPENDENCY_UNAVAILABLE
TIMEOUT
VALIDATION
AUTHORIZATION
PERMANENT
PANIC
CANCELLED
```

`PANIC` phải được recover bởi middleware, persist thành failed attempt và không được làm chết worker process.

Concurrency cần có nhiều levels thay vì một global setting:

Level	Ví dụ
Worker process	max 100 goroutines
Queue	max 40 concurrent
Function	max 5 concurrent
Tenant	max 20 concurrent
Resource group	max 10 <code>external-api-x</code> calls

`gocron` có `WithLimitConcurrentJobs` và `WithSingletonMode`, trong đó `LimitModeReschedule` bỏ qua execution khi limit đạt và schedule lần tiếp theo, còn `LimitModeWait` chờ slot. Những controls này nên dùng cho **scheduler's own lightweight internal jobs**, không dùng để thực hiện worker queue concurrency. ¹⁵

Rate limiting nên tách khỏi concurrency. Concurrency giới hạn “bao nhiêu executions chạy cùng lúc”; rate limit giới hạn “bao nhiêu executions được bắt đầu trong một khoảng thời gian”. Với distributed workers, Redis token bucket là một implementation phù hợp; tài liệu chính thức Redis có implementation token bucket bằng Redis/Lua cho Go và mô tả nó cho cả background-job throttling. ¹⁶

Policy có thể là:

```
{
  "algorithm": "TOKEN_BUCKET",
  "rate": 100,
  "period": "1m",
  "burst": 20,
  "scope": "QUEUE"
}
```

Batch processing nên hỗ trợ hai semantics riêng:

Bulk enqueue nghĩa là client tạo nhiều independent runs:

```
POST /api/v1/job-runs:bulk
```

Control DB tạo tất cả records và outbox events trong một transaction. Backend publication diễn ra async, vì vậy platform có thể đảm bảo **bulk submission durability** mà không cần backend hỗ trợ atomic multi-message publish.

Batch handler lại là một function invocation nhận nhiều records cùng lúc:

```
type BatchHandler func(
    ctx context.Context,
    items []JobPayload,
) []ItemResult
```

Batch result phải cho phép partial success để chỉ retry failed items nếu business semantics cho phép.

Middleware chain đề xuất:

```
Recover
↓
Trace
↓
Structured Logging
↓
Payload Decryption
↓
Idempotency
↓
Rate Limit
```

```

↓
Concurrency
↓
Timeout
↓
Custom Middleware
↓
Function Handler
↓
Metrics

```

Không nên cho UI upload executable middleware. Database chỉ lưu middleware configuration/binding; actual implementation được compile vào worker.

Interface:

```

type Handler func(context.Context, *Execution) error

type Middleware func(next Handler) Handler

```

Advanced Workflow Coordination nên là module riêng trên queue primitives thay vì cố gắng biểu diễn DAG bằng `gocron`. `gocron` giải quyết “when to invoke”, không phải persisted dependencies giữa business executions. Điều này là suy luận kiến trúc từ public Scheduler API, vốn tập trung vào independent scheduled jobs. ¹⁷

Workflow model đề xuất hỗ trợ:

```

A -----> C --> E
 \         /
  -> B -
      \
       -> D

```

Mỗi edge có condition:

```

ON_SUCCESS
ON_FAILURE
ALWAYS
EXPRESSION

```

Failure policies:

FAIL_FAST
CONTINUE
COMPENSATE
MANUAL_INTERVENTION

Workflow execution engine chỉ enqueue step khi dependency conditions đã satisfied; step vẫn sử dụng cùng JobRun/Attempt engine nên retry, DLQ, metrics và middleware không bị duplicate implementation.

Các NFR ban đầu dưới đây là **acceptance targets đề xuất**, cần benchmark lại theo workload thực tế:

NFR	Baseline target
Control-plane API availability	$\geq 99.9\%$
Metadata GET API	p95 < 250 ms
Single-job submission	p95 < 300 ms đến khi DB commit
Bulk 1,000 runs	p95 < 2 s đến khi durable trong control DB
Realtime UI propagation	p95 < 2 s
Schedule trigger lag	p99 < 2 s với minute-level Cron
Worker crash	Không mất accepted run
Queue backend outage	Accepted/outbox runs được publish lại sau recovery
Duplicate execution	Được phép theo at-least-once contract
Audit coverage	100% mutating administrative API
Retention	Configurable; 1 tháng / 3+ tháng theo tier
Deployment	Horizontal API/workers; HA scheduler
Graceful shutdown	Stop claim mới, chờ/cancel theo configured deadline

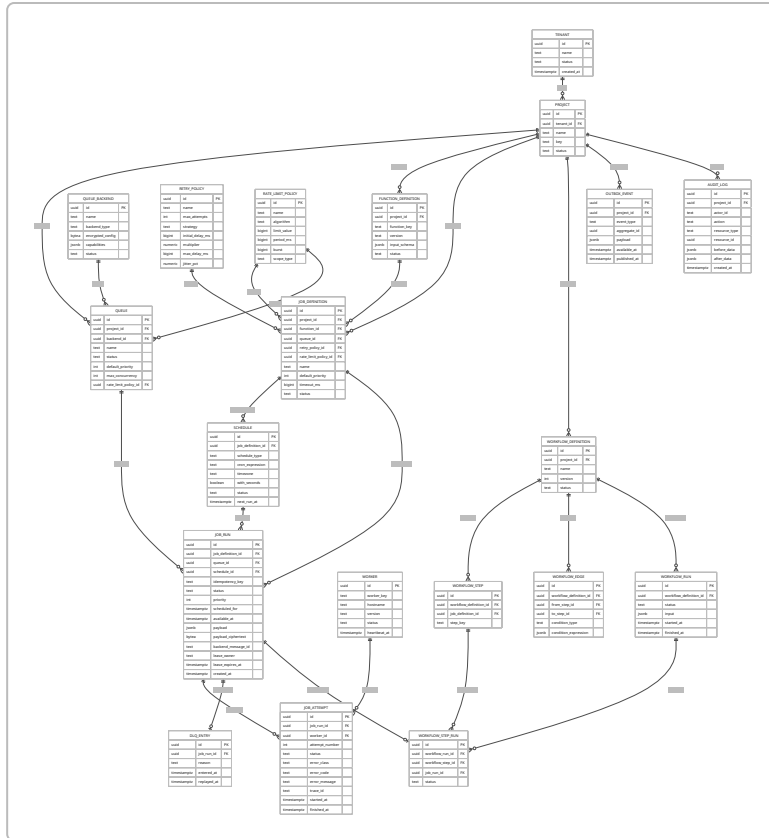
`gocron` v2 hiện có cả `ShutdownWithContext` và `StopJobsWithContext`, phù hợp với graceful lifecycle của scheduler service. ¹⁸

ERD và Data Model chi tiết

Core logical model nên ưu tiên PostgreSQL, UUID identifiers và timestamps dạng `timestampz`. Flexible metadata có thể dùng `jsonb`; PostgreSQL hỗ trợ GIN indexes để tìm keys/key-values trong `jsonb`, nhưng chỉ nên tạo GIN indexes cho fields thực sự được query để tránh write amplification không cần thiết.

¹⁹

ERD đề xuất:



Data dictionary cốt lõi:

Table	Vai trò	Những fields quan trọng
<code>tenants</code>	Security/billing boundary	<code>id</code> , <code>status</code>
<code>projects</code>	Namespace operational	<code>tenant_id</code> , <code>key</code>
<code>queue_backends</code>	Backend connection	<code>backend_type</code> , encrypted config, capabilities
<code>queues</code>	Logical queue	<code>status</code> , <code>backend</code> , <code>concurrency</code> , <code>priority</code>
<code>function_definitions</code>	Registry metadata	function key/version/schema
<code>job_definitions</code>	Reusable execution template	function, queue, timeout, policies
<code>schedules</code>	Recurring trigger	<code>cron</code> /type/timezone/status
<code>job_runs</code>	Logical execution	<code>status</code> , <code>payload</code> , <code>priority</code> , <code>timing</code> , <code>idempotency</code>
<code>job_attempts</code>	Physical executions	attempt number, worker, error, latency
<code>retry_policies</code>	Retry configuration	attempts/backoff/jitter

Table	Vai trò	Những fields quan trọng
<code>rate_limit_policies</code>	Throughput policy	algorithm/rate/burst
<code>dlq_entries</code>	Quarantined executions	reason/replay tracking
<code>workers</code>	Runtime instances	heartbeat/version/status
<code>workflow_*</code>	DAG definitions/runtime	steps/edges/runs
<code>outbox_events</code>	Reliable asynchronous publishing	available/published state
<code>audit_logs</code>	Admin audit trail	actor/action/before/after

`job_runs` phải giữ **snapshot của execution-critical configuration**. Nếu RetryPolicy của JobDefinition được thay đổi khi một run đang retry, behavior của run cũ không nên âm thầm đổi. Cách đơn giản là lưu:

```
policy_snapshot jsonb NOT NULL
```

chứa:

```
{
  "retry": {
    "max_attempts": 5,
    "strategy": "EXPONENTIAL",
    "initial_delay_ms": 1000
  },
  "timeout_ms": 30000,
  "rate_limit_policy_version": 3
}
```

Tương tự, `function_version` nên snapshot vào `job_runs`. Điều này làm cho execution history reproducible.

Các indexes quan trọng:

```
CREATE UNIQUE INDEX ux_job_runs_idempotency
ON job_runs(project_id, idempotency_key)
WHERE idempotency_key IS NOT NULL;
```

```
CREATE UNIQUE INDEX ux_schedule_occurrence
ON job_runs(schedule_id, scheduled_for)
WHERE schedule_id IS NOT NULL;
```

```
CREATE INDEX ix_job_runs_claim
ON job_runs(
    queue_id,
    priority DESC,
    available_at ASC,
    id ASC
)
WHERE status = 'QUEUED';
```

```
CREATE INDEX ix_job_runs_retry
ON job_runs(available_at)
WHERE status = 'RETRY_WAIT';
```

```
CREATE INDEX ix_attempts_run
ON job_attempts(job_run_id, attempt_number);
```

```
CREATE INDEX ix_dlq_open
ON dlq_entries(entered_at DESC)
WHERE replayed_at IS NULL;
```

PostgreSQL partial indexes chứa chỉ những rows đáp ứng predicate, nên phù hợp cho hot operational subsets như `QUEUED`, `RETRY_WAIT` hoặc unreplayed DLQ thay vì index toàn bộ lịch sử completed runs.

20

Khi `job_runs` và `job_attempts` đạt volume lớn, history có thể được declaratively partition theo `created_at` / `started_at` theo tháng. PostgreSQL hỗ trợ range, list và hash declarative partitioning.

21

Một hướng partitioning:

```
CREATE TABLE job_attempts (
    ...
    started_at timestamptz NOT NULL
) PARTITION BY RANGE (started_at);
```

```
job_attempts_2026_08
job_attempts_2026_09
job_attempts_2026_10
...
```


Từ đó retention 1 tháng hoặc 3+ tháng có thể thực hiện bằng detach/drop old partitions nhanh hơn xóa từng hàng, nhưng chỉ nên áp dụng sau khi volume thực tế chứng minh cần partitioning.

Với payload encryption, PostgreSQL có `pgcrypto` và hỗ trợ cryptographic functions cũng như symmetric/public-key encryption cho từng field. PostgreSQL cũng mô tả `pgcrypto` là một option cho column-level encryption. ²² Tuy nhiên BRD nên abstraction hóa thành `EncryptionProvider` để production có thể dùng application-level envelope encryption với external KMS/HSM thay vì khóa mã hóa nằm cùng database.

Schema gợi ý:

```
payload          jsonb NULL,
payload_ciphertext bytea NULL,
encryption_key_ref text NULL,
```

với constraint:

```
CHECK (
    (payload IS NOT NULL AND payload_ciphertext IS NULL)
    OR (payload IS NULL AND payload_ciphertext IS NOT NULL)
)
```

Sensitive data không nên được duplicate sang logs, audit `before_data/after_data` hoặc Prometheus labels.

API, Web UI, observability và security model

REST API nên phân biệt resource definition và execution.

Method	Endpoint	Purpose
POST	<code>/api/v1/job-definitions</code>	Tạo job definition
GET	<code>/api/v1/job-definitions</code>	Search definitions
POST	<code>/api/v1/job-definitions/{id}/run</code>	Tạo immediate run
POST	<code>/api/v1/job-runs:bulk</code>	Bulk submit
GET	<code>/api/v1/job-runs</code>	Search history
GET	<code>/api/v1/job-runs/{id}</code>	Run detail
POST	<code>/api/v1/job-runs/{id}/cancel</code>	Cancel
POST	<code>/api/v1/job-runs/{id}/retry</code>	Operator retry
GET	<code>/api/v1/job-runs/{id}/attempts</code>	Attempts

Method	Endpoint	Purpose
POST	/api/v1/schedules	Create schedule
PATCH	/api/v1/schedules/{id}	Update
POST	/api/v1/schedules/{id}/pause	Pause
POST	/api/v1/schedules/{id}/resume	Resume
POST	/api/v1/queues	Create queue
POST	/api/v1/queues/{id}/pause	Soft pause
POST	/api/v1/queues/{id}/resume	Resume
POST	/api/v1/queues/{id}/drain	Drain
GET	/api/v1/dlq	Search DLQ
POST	/api/v1/dlq/{id}/replay	Replay one
POST	/api/v1/dlq:replay	Bulk replay
GET	/api/v1/workers	Worker health
POST	/api/v1/workflows	Create workflow
POST	/api/v1/workflows/{id}/run	Execute DAG
GET	/api/v1/audit-logs	Audit search

HTTP errors nên chuẩn hóa theo **RFC 9457 Problem Details for HTTP APIs**, là RFC hiện hành thay thế RFC 7807. ²³ Ví dụ:

```
{
  "type": "urn:task-platform:error:queue-paused",
  "title": "Queue is paused",
  "status": 409,
  "detail": "Queue billing-high is currently paused",
  "instance": "/api/v1/job-runs/...",
  "code": "QUEUE_PAUSED",
  "retryable": true
}
```

Bulk API nên trả về `batch_id` và individual run IDs, ví dụ `202 Accepted`, vì publication tới backend xảy ra async qua outbox.

Realtime API có thể dùng:

```
WebSocket: /api/v1/events/ws
```

hoặc SSE:

```
GET /api/v1/events
```

Events tối thiểu:

```
job.created  
job.queued  
job.started  
job.retry_scheduled  
job.succeeded  
job.failed  
job.dlq  
queue.paused  
queue.resumed  
worker.online  
worker.offline  
schedule.changed  
workflow.changed
```

`gocron-ui` hiện dùng WebSocket cho live updates và hỗ trợ monitoring nhiều scheduler instances, chứng minh đây là integration pattern phù hợp với `gocron`; tuy nhiên authentication phải được platform bổ sung. ⁵

UI đề xuất gồm các views:

View	Nội dung
Dashboard	throughput, failure rate, queue depth, running workers, DLQ
Queues	backlog, oldest job, priority, pause/drain
Job Definitions	function, policies, schedules
Runs	searchable run history
Run Detail	payload metadata, attempts, logs, trace
Schedules	Cron preview, next executions, enable/pause
DLQ	reason grouping, inspect, replay
Workflows	DAG visualization + run progress
Workers	heartbeat, version, inflight jobs

View	Nội dung
Metrics	operational charts
Alerts	active/acknowledged alerts
Settings	backends, retry/rate policies
Audit	who changed what and when

RBAC:

Permission	Admin	Operator	Developer	Viewer
Read jobs/history	✓	✓	✓	✓
Run job	✓	✓	✓	—
Create definitions	✓	—	✓	—
Change schedule	✓	✓	✓	—
Pause queue	✓	✓	—	—
DLQ replay	✓	✓	—	—
Configure backend	✓	—	—	—
Manage RBAC	✓	—	—	—
Read audit	✓	✓	—	optional

Mọi API query phải automatically scope bằng:

```
tenant_id
└─ project_id
    └─ resource
```

Không nên dựa vào frontend để filter tenant; tenant/project authorization phải được enforce server-side.

Observability nên dựa trên **OpenTelemetry + Prometheus-compatible metrics**. OpenTelemetry Go hiện đánh dấu traces và metrics là Stable, logs là Beta; OpenTelemetry cung cấp vendor-neutral path để generate traces/metrics/logs và Collector có thể route telemetry tới nhiều backends. ²⁴

`gocron` cũng cung cấp `Monitor` / `MonitorStatus` interfaces để collect job counts, timing, status và error, cùng event listeners trước/sau execution. Đây nên được dùng cho scheduler metrics; worker metrics vẫn do task platform tự instrument. ²⁵

Metric catalog:

```

task_runs_created_total
task_runs_completed_total
task_runs_failed_total
task_attempts_total
task_retries_total
task_dlq_total

task_attempt_duration_seconds
task_queue_wait_seconds
task_schedule_lag_seconds

task_queue_depth
task_queue_oldest_age_seconds

task_worker_inflight
task_workers_online

task_outbox_pending
task_outbox_oldest_age_seconds

```

Prometheus định nghĩa counters phù hợp cho cumulative events, gauges cho values có thể lên/xuống như concurrent requests hoặc queue size, và histograms cho distributions như latency. ²⁶ Vì vậy mapping đề xuất là:

*_total	→ Counter
queue_depth	→ Gauge
worker_inflight	→ Gauge
*_duration_seconds	→ Histogram
*_wait_seconds	→ Histogram

Không nên đưa `job_run_id`, `user_id` hay error message vào Prometheus labels. Prometheus instrumentation guidance cảnh báo label cardinality có thể tăng thành hàng triệu time series; identifiers có cardinality cao nên nằm trong logs/traces thay vì metric dimensions. ²⁷

Recommended metric labels chỉ nên có bounded dimensions:

```

project
queue
function
status
error_class
worker_pool
backend_type

```

OpenTelemetry trace nên liên kết:

```
HTTP request
├─ create-job-run
│   └─ outbox-publish
│       └─ queue-reserve
│           └─ job-attempt
│               └─ middleware
│                   └─ function-handler
│                       └─ external dependency
```

OpenTelemetry hỗ trợ correlation logs/traces qua TraceId và SpanId, vì vậy `job_attempts.trace_id` tạo đường nối trực tiếp giữa Web UI execution history và distributed trace backend. ²⁸

Retention nên tách thành:

```
Job execution metadata → PostgreSQL
Application logs        → log backend
Metrics                 → Prometheus-compatible TSDB
Traces                  → tracing backend
Audit                   → PostgreSQL / immutable archive
```

Không nên lưu full verbose application logs hàng tháng trong `job_attempts.error_message`. Database chỉ lưu normalized failure summary và pointers tới external logs/traces.

Roadmap triển khai, acceptance criteria và các rủi ro chính

Lộ trình tốt nhất là không cố build tất cả enterprise capabilities trong một release.

Foundation release nên xây Postgres control DB, PostgreSQL queue backend, Job Definition/Run/Attempt separation, gocron scheduler, outbox, retries, DLQ, concurrency, priority, pause/resume, worker heartbeat, REST API, authentication/RBAC, realtime dashboard, basic Prometheus metrics và audit. PostgreSQL `SKIP LOCKED` có semantics phù hợp với queue-like consumers nên đây là backend khởi đầu ít thành phần nhất.

²⁹

Scale release bổ sung Redis Streams và/hoặc NATS JetStream adapters, distributed token-bucket rate limiting, batch handler, optimized bulk enqueue, autoscaling metrics, OpenTelemetry Collector integration và custom telemetry exporters. Redis Streams và JetStream đều có durable consumer semantics phù hợp cho horizontally scaled workers. ³⁰

Enterprise release bổ sung workflows/DAG, encrypted sensitive payloads/KMS integration, tenant isolation enhancements, configurable retention, richer scoped UI, policy administration, multi-region strategy và commercial support/SLA functions.

Recommended Go code layout:

```
cmd/  
  api/  
  scheduler/  
  worker/  
  
internal/  
  domain/  
    job/  
    queue/  
    retry/  
    workflow/  
  
  application/  
    jobs/  
    scheduling/  
    workers/  
  
  queue/  
    backend.go  
    postgres/  
    redisstreams/  
    jetstream/  
  
  scheduler/  
    gocron/  
  
  worker/  
    executor/  
    middleware/  
    registry/  
  
  persistence/  
    postgres/  
  
  telemetry/  
    otel/  
    prometheus/  
  
  security/  
    auth/  
    rbac/  
    encryption/  
  
  realtime/  
    websocket/
```

outbox/

Điểm design quan trọng là package:

internal/domain

không được import Redis/NATS/gocron-specific types. Domain phải độc lập khỏi infrastructure để backend thay đổi mà không rewrite business rules.

Acceptance suite đề xuất:

Area	Acceptance test
Durability	Submit 10,000 jobs, kill API/worker/scheduler processes ngẫu nhiên; mọi accepted run vẫn ở terminal/retry/DLQ state
Duplicate safety	Crash worker sau business side effect nhưng trước ACK; idempotency ngăn duplicate side effect
Scheduler HA	Kill active scheduler; standby nhận leader role và không tạo duplicate (schedule_id, scheduled_for)
Retry	Inject transient errors; verify backoff/max-attempt/jitter
Permanent failure	Validation error không retry, chuyển đúng terminal/DLQ policy
Lease recovery	Kill worker đang RUNNING; expired lease được worker khác recover
Queue pause	Pause không reserve mới; in-flight executions được hoàn thành
Drain	Reject/redirect enqueue mới và xử lý hết backlog
Priority	HIGH ưu tiên hơn LOW nhưng LOW không starvation vô hạn
Rate limiting	Multiple workers vẫn tổng cộng không vượt configured distributed rate
Bulk	Commit 1,000 runs rồi kill dispatcher; restart vẫn publish đủ qua outbox
DLQ	Exhaust retries → DLQ → replay → execution trở lại normal lifecycle
Authentication	Anonymous caller không truy cập operational API
Tenant isolation	Tenant A không enumerate/read Tenant B IDs
Audit	Mọi create/update/delete/pause/replay đều sinh audit record
Realtime	UI disconnect/reconnect không làm state sai; REST refetch phục hồi
Telemetry	Run → attempt → trace/log correlation được resolve

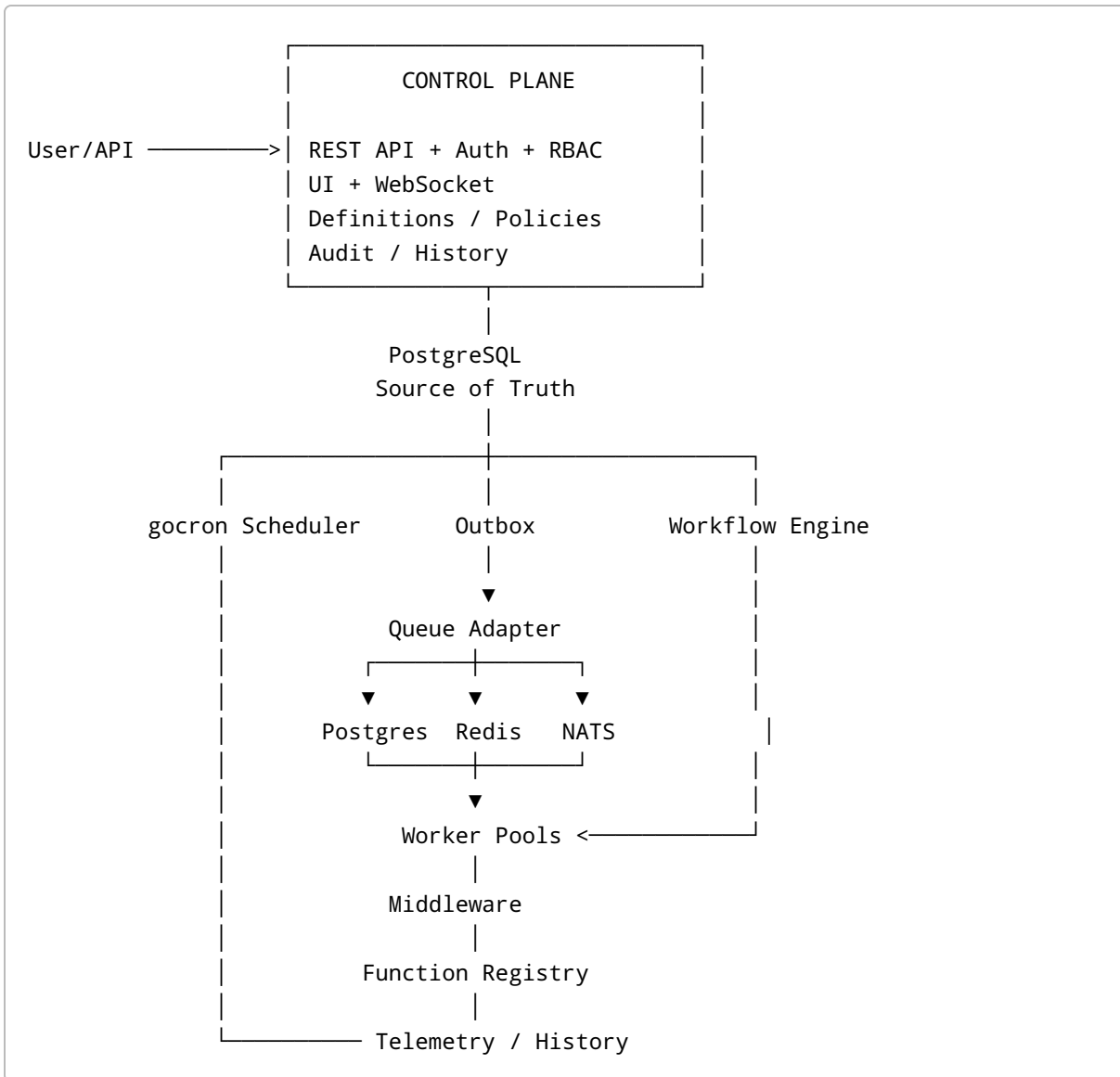
Area	Acceptance test
Backend parity	Contract suite chạy chung trên PostgreSQL, Redis, JetStream adapters

Risk register quan trọng nhất:

Risk	Impact	Mitigation
Dùng <code>gocron</code> như durable queue	Job backlog nằm trong process, semantics không đủ	Chỉ dùng <code>gocron</code> cho scheduling
<code>LimitModeWait</code> bị backlog	Execution ordering khó dự đoán	Durable queue riêng; tránh dùng Wait cho business tasks
Duplicate scheduled occurrence	Business function chạy hai lần	Elector + unique (<code>schedule_id</code> , <code>scheduled_for</code>)
Worker crash sau side effect	At-least-once gây duplicate	Idempotency key/business dedup
Backend semantics khác nhau	Behavior không nhất quán	Capability-based adapter contract
Priority starvation	Low priority không bao giờ chạy	Priority bands + weighted fair polling
Clock skew scheduler	Trigger distribution sai	Leader election + clock monitoring
Queue DB phình lớn	Claim/history query chậm	Partial indexes + archive/partition strategy
Retry storm	Dependency outage làm tăng tải	Exponential backoff + jitter + rate limiter/circuit policy
Poison job	Retry vô hạn	max attempts + DLQ
DLQ không được xử lý	Silent data loss về business	DLQ age/depth alerts
Metrics cardinality	Prometheus memory explosion	Không dùng run IDs/user IDs làm labels
Payload chứa secrets	Leakage qua logs/history	Encryption + redaction + access policy
Arbitrary code qua UI	Security/RCE	Compile-time function registry
WebSocket mất event	UI stale	Events là hint; REST/DB luôn là source of truth
Queue backend outage	Submission thất bại	Transactional outbox
Policy thay đổi giữa retry	Execution không reproducible	Snapshot policies trên JobRun

Hai risk đầu tiên được chính tài liệu `gocron` củng cố: package định vị chính nó là scheduler và `LimitModeWait` có cảnh báo về piled-up jobs/order không đảm bảo; distributed locker cũng không tự đồng bộ run times và có clock-skew considerations. ³¹

Kiến trúc cuối cùng nên được xem như sau:



Với ranh giới này, **gocron** **đảm nhiệm phần nó mạnh nhất: scheduling**, còn platform tự bổ sung durable execution engine cho Job Queues, retry, priority, rate limiting, DLQ, workflows, history, scoped UI và enterprise controls. Đây cũng là cách tránh phụ thuộc quá sâu vào behavior của một scheduler library trong khi vẫn tận dụng đầy đủ Cron, event listeners, distributed election/locking, monitors và lifecycle APIs mà **gocron** v2 hiện cung cấp. ³²

¹ ¹⁸ Releases · go-co-op/gocron · GitHub

<https://github.com/go-co-op/gocron/releases>

² ³¹ GitHub - go-co-op/gocron: Easy and fluent Go cron scheduling. This is a fork from <https://github.com/jasonlvhit/gocron> · GitHub

<https://github.com/go-co-op/gocron>

3 4 6 12 13 14 15 17 25 32 **gocron package - github.com/go-co-op/gocron/v2 - Go Packages**

<https://pkg.go.dev/github.com/go-co-op/gocron/v2>

5 **GitHub - go-co-op/gocron-ui: A lightweight, real-time web interface for monitoring and controlling gocron scheduled jobs. It provides a complete solution for visualizing job schedules, tracking execution status, and managing jobs through an intuitive web dashboard. · GitHub**

<https://github.com/go-co-op/gocron-ui>

7 29 **PostgreSQL: Documentation: 18: SELECT**

https://www.postgresql.org/docs/current/sql-select.html?utm_source=chatgpt.com

8 **Documentation: 18: 13.3. Explicit Locking**

https://www.postgresql.org/docs/current/explicit-locking.html?utm_source=chatgpt.com

9 **Documentation: 18: NOTIFY**

https://www.postgresql.org/docs/current/sql-notify.html?utm_source=chatgpt.com

10 **Redis Streams | Docs**

https://redis.io/docs/latest/develop/data-types/streams/?utm_source=chatgpt.com

11 **JetStream - NATS Documentation**

https://docs.nats.io/concepts/jetstream?utm_source=chatgpt.com

16 **Token bucket rate limiter with Redis and Go | Docs**

https://redis.io/docs/latest/develop/use-cases/rate-limiter/go/?utm_source=chatgpt.com

19 **Documentation: 18: 65.4. GIN Indexes**

https://www.postgresql.org/docs/current/gin.html?utm_source=chatgpt.com

20 **Documentation: 18: 11.8. Partial Indexes**

https://www.postgresql.org/docs/current/indexes-partial.html?utm_source=chatgpt.com

21 **Documentation: 18: 5.12. Table Partitioning**

https://www.postgresql.org/docs/current/ddl-partitioning.html?utm_source=chatgpt.com

22 **Documentation: 18: F.26. pgcrypto — cryptographic functions**

https://www.postgresql.org/docs/current/pgcrypto.html?utm_source=chatgpt.com

23 **RFC 9457 - Problem Details for HTTP APIs**

https://datatracker.ietf.org/doc/html/rfc9457?utm_source=chatgpt.com

24 **Go**

https://opentelemetry.io/docs/languages/go/?utm_source=chatgpt.com

26 **Metric types**

https://prometheus.io/docs/concepts/metric_types/?utm_source=chatgpt.com

27 **Instrumentation**

https://prometheus.io/docs/practices/instrumentation/?utm_source=chatgpt.com

28 **OpenTelemetry Logging**

https://opentelemetry.io/docs/specs/otel/logs/?utm_source=chatgpt.com

30 **Redis streaming with go-redis | Docs**

https://redis.io/docs/latest/develop/use-cases/streaming/go/?utm_source=chatgpt.com